PX4 Prelim Guidelines
Ryan Mahmoudi

MAVSDK Python QuickStart

- Use 64-bit Raspberry Pi OS and a Python virtual environment.
- Current MAVSDK supports Linux aarch64 wheels, including 64-bit Pi OS.
- Install mavsdk; pin the tested package version and Python version.
- Current async imports use mavsdk.asyncio and ActionAsync.
- Connect, discover the autopilot, then create its action plugin.
- MAVSDK 3.x examples use a different API; keep code and docs aligned.

PX4 Gazebo simulation

- Run PX4 SITL and Gazebo on a development computer.
- Launch: make px4_sitl gz_x500.
- Connect the Pi test application over the network; monitor in QGroundControl.
- SITL commonly sends SDK traffic to UDP 14540; configure the Pi endpoint.
- Stock X500 validates sequencing; model the custom chassis for performance.

Raspberry Pi UART configuration

- Choose USB serial or a configured 3.3 V UART connection.
- For GPIO UART: cross TX/RX, share ground, and verify connector pinouts.
- Pi 5 primary UART defaults to its debug header; select the intended interface.
- Disable serial console use; match baud and check device permissions.
- Store transport, device path, and baud in configuration.

Pixhawk connection reference

- Configure the selected flight-controller port for MAVLink.
- Verify heartbeat and vehicle identity before sending commands.
- Pi sends high-level commands; Pixhawk drives ESC outputs.
- Keep power regulation separate from the data connection.

Test sequence and telemetry

MAVSDK takeoff and landing example

- Legacy API example: use its sequence, or pin its compatible SDK.
- States: connect → ready → arm → climb → hover → land → disarmed.
- Confirm each transition through fresh telemetry.
- Command acceptance does not prove altitude or touchdown.
- Replace fixed sleeps with measured conditions and deadlines.
- Adapt global-position readiness checks for validated indoor local positioning.

PX4 Takeoff mode

- Automatic takeoff requires valid local position.
- Configure target altitude and climb speed within fixture clearance.
- Review MIS_TAKEOFF_ALT and MPC_TKO_SPEED.
- Proposed climb check: height reaches target tolerance before timeout.
- Proposed hover check: height, drift, and attitude stay within agreed limits.

PX4 Land mode

- Land descends at the location where the mode starts.
- Review MPC_LAND_SPEED and COM_DISARM_LAND.
- Watch landing state, vertical motion, and armed status.
- Require touchdown confirmation and disarming before reporting success.
- On timeout, record failure and invoke the tested recovery procedure.

PX4 Prelim Guidelines
Ryan Mahmoudi

PX4 optical flow integration

- Indoor option: downward optical flow plus distance sensing.
- A rangefinder alone cannot establish horizontal position.
- Check local-position validity before arming and throughout flight.
- Use one consistent local frame; NED down is positive.
- Compute climb from the starting position; detect estimator resets.
- Log height, XY drift, attitude, battery, mode, link, and landing state.

Reliability and verification

Python asyncio tasks and timeouts

- Run telemetry collection, sequencing, and watchdogs concurrently.
- Use asyncio.wait_for() for bounded waits; use a monotonic clock.
- Timestamp samples; reject stale data even while commands succeed.
- Catch command errors; record state and reason before recovery.
- Use try/finally for cleanup; cancelling Python does not stop the aircraft.
- Proposed modules: connection, telemetry, sequence, watchdog, logger.

PX4 failsafes

- Configure PX4 link-loss, position-loss, and battery responses.
- A dead Pi cannot send a recovery command; PX4 must handle that case.
- Keep RC takeover and emergency motor stop independent of Pi.
- Test kill behavior without props; releasing within five seconds can restart motors.
- Proposed config: altitude, speed, drift/tilt limits, timeouts, sample-age limits.

PX4 failsafe simulation

- Test missing readiness, rejected commands, and unreachable target altitude.
- Test telemetry loss, Pi process termination, and position failure in SITL.
- Check both application decisions and the actual PX4 failsafe response.
- Advance: SITL → prop-free integration → supervised restrained flight.
- Keep restraints unloaded in hover; allow actual landing-gear touchdown.

PX4 Flight Review

- Save PX4 ULog plus timestamped application CSV or JSON logs.
- Record commands, responses, transitions, telemetry, and failure reasons.
- Archive PX4 parameters, SDK version, code commit, mass, and battery state.
- Compare commanded height with measured height, drift, and motor outputs.
- Proposed pass: complete sequence, no faults, touchdown, and disarming.
- Require three consecutive passing runs before expanding test limits.

PX4 Offboard mode

- Optional later step for explicit vertical setpoints; start with Takeoff/Land.
- PX4 requires a >2 Hz proof-of-life stream, established for over one second.
- Check COM_OF_LOSS_T and configured offboard-loss action.
- Validate setpoint frame, sign, and loss response in SITL first.